

David_图论

David_图论

普通图论

Tarjan缩点求最大连通分量

倍增LCA

树链剖分（独立版）

树链剖分（结构体版）

动态维护树直径、端点、中心

树的直径

树的重心

Bellman-Ford（有边数限制最短路）

SPFA判断负环

SPFA求最短路

普通Dijkstra求最短路

堆优化Dijkstra求最短路

Floyd求多源最短路

Prim算法求最小生成树

Kruskal算法求最小生成树

拓扑排序

匈牙利二分图最大匹配

带权二分图最优匹配（KM算法）

染色法判定二分图

网络流

Dinic最大流

EK最大流

Dinic最小割

最小费用最大流

带费用上下界最小费用可行循环流

普通图论

Tarjan缩点求最大连通分量

```
1  #define int long long
2  using namespace std;
3  int mod=1e9+7;
4  int MAXN=5e5+5;
5  vector<vector<int>>> gra(MAXN); //原图邻接表
6  vector<int> val(MAXN); //原图点权
7  vector<int> dfn(MAXN); //原图dfn序，用于构建scc
8  vector<int> low(MAXN); //节点的scc根节点，用于构建scc
9  stack<int> stk; //栈，用于构建scc
10 vector<int> instk(MAXN); //节点是否入栈，用于构建scc
11 vector<int> itscc(MAXN); //ind to scc 原图点角标转新图点角标
12 vector<int> sccval(MAXN); //新图点权
13 vector<vector<int>>> sccgra(MAXN); //新图邻接表
14 int cur=1; //维护原图dfn值
15 int curscc=0; //新图大小
16
17 int n,m;
18 vector<pair<int,int>>> edge(MAXN);
```

```

19 void tarjan(int ind){
20     dfn[ind]=cur;
21     low[ind]=cur;
22     cur++;
23     instk[ind]=1;
24     stk.push(ind);
25     for(int i:gra[ind]){
26         //无向图这里加 if(par[i]==ind)continue;
27         if(!dfn[i]){
28             tarjan(i);
29             low[ind]=min(low[ind],low[i]);
30         }else if(instk[i]){
31             low[ind]=min(low[ind],dfn[i]);
32         }
33     }
34
35     if(low[ind]==dfn[ind]){
36         int cval=0;
37         int ctar=0;
38         int c;
39         do{
40             c=stk.top();
41             stk.pop();
42             instk[c]=0;
43             itscc[c]=curscc;
44             sccval[curscc]+=val[c];
45         }while(c!=ind);
46         curscc++;
47     }
48 }
49
50 void build(){
51     //第一步求解scc
52     for(int i=1;i<=n;i++){
53         if(!dfn[i])
54             tarjan(i);
55     }
56     //第二步建新图
57     for(int i=0;i<m;i++){
58         int a=edge[i].first;
59         int b=edge[i].second;
60         if(itscc[a]!=itscc[b]){
61             sccgra[itscc[a]].push_back(itscc[b]);
62         }
63     }
64 }

```

倍增LCA

```

1  // #define int long long
2  const int N=4e4+10;
3  int n,a,b,m,x,y;
4  vector<int> arr[N];
5  int depth[N],fa[N][16];
6
7  void bfs(int root)
8  {
9      memset(depth,0x3f,sizeof depth);
10     depth[0]=0,depth[root]=1;
11     queue<int> q;

```

```

12     q.push(root);
13     while(!q.empty())
14     {
15         int t=q.front();
16         q.pop();
17         for(auto j:arr[t])
18         {
19             if(depth[j]>depth[t]+1)
20             {
21                 depth[j]=depth[t]+1;
22                 q.push(j);
23                 fa[j][0]=t;//j往上跳一步是t
24                 for(int k=1;k<=15;k++)
25                 {
26                     fa[j][k]=fa[fa[j][k-1]][k-1];
27                 }
28             }
29         }
30     }
31 }
32
33 int lca(int a,int b)
34 {
35     if(depth[a]<depth[b])
36     {
37         swap(a,b);
38     }
39     for(int k=15;k>=0;k--)
40     {
41         if(depth[fa[a][k]]>=depth[b])
42         {
43             a=fa[a][k];
44         }
45     }
46     if(a==b)
47         return a;
48     for(int k=15;k>=0;k--)
49     {
50         if(fa[a][k]!=fa[b][k])
51         {
52             a=fa[a][k];
53             b=fa[b][k];
54         }
55     }
56     return fa[a][0];
57 }

```

树链剖分（独立版）

```

1  #define int long long
2  const int N=1e5+10;
3  int n,m,root,mod,x,y,z,op;
4  vector<int> arr[N];
5  int w[N],fa[N],sz[N],son[N],dep[N],top[N],id[N],nw[N];
6  int tot;
7
8  void dfs1(int u,int father)
9  {
10     fa[u]=father,sz[u]=1,dep[u]=dep[father]+1;
11     for(auto v:arr[u])

```

```

12     {
13         if(v==father) continue;
14         dfs1(v,u);
15         sz[u]+=sz[v];
16         if(sz[son[u]]<sz[v])
17         {
18             son[u]=v;
19         }
20     }
21 }
22
23 void dfs2(int u,int t)
24 {
25     top[u]=t,id[u]++;tot,nw[tot]=w[u]%mod;
26     if(!son[u]) return;
27     dfs2(son[u],t);
28     for(auto v:arr[u])
29     {
30         if(v==fa[u] || v==son[u])
31         {
32             continue;
33         }
34         dfs2(v,v);
35     }
36 }
37
38 struct tree
39 {
40     int l,r,sum,lazy;
41 }tr[4*N];
42
43 void pushup(int u)
44 {
45     tr[u].sum=(tr[u*2].sum+tr[u*2+1].sum)%mod;
46 }
47
48 void pushdown(int u)
49 {
50     if(tr[u].lazy)
51     {
52         tr[u*2].lazy=(tr[u*2].lazy+tr[u].lazy)%mod;
53         tr[u*2+1].lazy=(tr[u*2+1].lazy+tr[u].lazy)%mod;
54         tr[u*2].sum=(tr[u*2].sum+tr[u].lazy*(tr[u*2].r-tr[u*2].l+1))%mod;
55         tr[u*2+1].sum=(tr[u*2+1].sum+tr[u].lazy*(tr[u*2+1].r-tr[u*2+1].l+1))%mod;
56         tr[u].lazy=0;
57     }
58 }
59
60 void build(int u,int l,int r)
61 {
62     tr[u].l=l;
63     tr[u].r=r;
64     tr[u].lazy=0;
65     if(l==r)
66     {
67         tr[u].sum=nw[l]%mod;
68         return;
69     }
70     int mid=l+r>>1;
71     build(u*2,l,mid);
72     build(u*2+1,mid+1,r);

```

```

73     pushup(u);
74 }
75
76 int query(int u,int l,int r)
77 {
78     if(l<=tr[u].l&&tr[u].r<=r)
79     {
80         return tr[u].sum%mod;
81     }
82     int mid=tr[u].l+tr[u].r>>1,res=0;
83     pushdown(u);
84     if(l<=mid) res=(res+query(u*2,l,r))%mod;
85     if(mid<r) res=(res+query(u*2+1,l,r))%mod;
86
87     return res%mod;
88 }
89
90 int query_path(int u,int v)
91 {
92     int res=0;
93     while(top[u]!=top[v])
94     {
95         if(dep[top[u]]<dep[top[v]]) swap(u,v);
96         res=(res+query(1,id[top[u]],id[u]))%mod;
97         u=fa[top[u]];
98     }
99     if(dep[u]<dep[v]) swap(u,v);
100    res=(res+query(1,id[v],id[u]))%mod;
101    return res%mod;
102 }
103
104 void update(int u,int l,int r,int k)
105 {
106     k%=mod;
107     if(l<=tr[u].l&&tr[u].r<=r)
108     {
109         tr[u].lazy=(tr[u].lazy+k)%mod;
110         tr[u].sum=(tr[u].sum+k*(tr[u].r-tr[u].l+1))%mod;
111         return;
112     }
113     pushdown(u);
114     int mid=tr[u].l+tr[u].r>>1;
115     if(l<=mid) update(u*2,l,r,k);
116     if(mid<r) update(u*2+1,l,r,k);
117     pushup(u);
118 }
119
120 void update_path(int u,int v,int k)
121 {
122     while(top[u]!=top[v])
123     {
124         if(dep[top[u]]<dep[top[v]]) swap(u,v);
125         update(1,id[top[u]],id[u],k);
126         u=fa[top[u]];
127     }
128     if(dep[u]<dep[v]) swap(u,v);
129     update(1,id[v],id[u],k);
130 }
131
132 void update_tree(int u,int k)
133 {

```

```

134     update(1, id[u], id[u]+sz[u]-1, k);
135 }
136
137 int query_tree(int u)
138 {
139     return query(1, id[u], id[u]+sz[u]-1)%mod;
140 }

```

树链剖分（结构体版）

```

1  // #include "../数据结构/线段树.cpp"
2
3  //线段树(辅助树链剖分)
4  struct Seg{
5      int n;
6      vector<int> tr, lazy;
7
8      Seg(int n) : n(n), tr(n * 4 + 5), lazy(n * 4 + 5) {}
9
10     void build(int p, int l, int r, const vector<int>& a){
11         if(l == r){
12             tr[p] = a[l];
13             return;
14         }
15         int mid = l + r >> 1;
16         build(p << 1, l, mid, a);
17         build(p << 1 | 1, mid + 1, r, a);
18         pushup(p);
19     }
20
21     void clear(){
22         fill(tr.begin(), tr.end(), 0);
23         fill(lazy.begin(), lazy.end(), 0);
24     }
25
26     void pushup(int p){
27         tr[p] = tr[p << 1] + tr[p << 1 | 1];
28     }
29
30     void pushdown(int p, int l, int r){
31         if(lazy[p]){
32             int mid = l + r >> 1;
33             tr[p << 1] += (mid - l + 1) * lazy[p];
34             tr[p << 1 | 1] += (r - mid) * lazy[p];
35             lazy[p << 1] += lazy[p];
36             lazy[p << 1 | 1] += lazy[p];
37             lazy[p] = 0;
38         }
39     }
40
41     void update(int p, int l, int r, int ql, int qr, int val){
42         if(ql <= l && r <= qr){
43             tr[p] += (r - l + 1) * val;
44             lazy[p] += val;
45             return;
46         }
47         pushdown(p, l, r);
48         int mid = l + r >> 1;
49         if(ql <= mid) update(p << 1, l, mid, ql, qr, val);
50         if(mid < qr) update(p << 1 | 1, mid + 1, r, ql, qr, val);

```

```

51     pushup(p);
52 }
53
54 int ask(int p, int l, int r, int ql, int qr){
55     if(ql <= l && r <= qr){
56         return tr[p];
57     }
58     pushdown(p, l, r);
59     int mid = l + r >> 1;
60     int res = 0;
61     if(ql <= mid) res += ask(p << 1, l, mid, ql, qr);
62     if(mid < qr) res += ask(p << 1 | 1, mid + 1, r, ql, qr);
63     return res;
64 }
65
66 };
67
68 //树链剖分
69 struct HLD{
70     int n, timer;
71     vector<int> sz, dep, fa, son, top, dfn, rnk;
72     vector<vector<int>> adj;
73     Seg seg;
74
75     HLD(int n) : n(n), timer(0), sz(n + 1), dep(n + 1), fa(n + 1), son(n + 1),
76     top(n + 1), dfn(n + 1), rnk(n + 1), adj(n + 1), seg(n) {}
77
78     void addEdge(int u, int v){
79         adj[u].push_back(v);
80         adj[v].push_back(u);
81     }
82
83     //处理 sz, dep, fa, son
84     void dfs1(int u, int p, int d){
85         sz[u] = 1; fa[u] = p; dep[u] = d; son[u] = 0;
86         for(int v : adj[u]){
87             if(v == p) continue;
88             dfs1(v, u, d + 1);
89             sz[u] += sz[v];
90             if(sz[v] > sz[son[u]]) son[u] = v;
91         }
92     }
93
94     //处理 top, dfn
95     void dfs2(int u, int tp){
96         top[u] = tp; dfn[u] = ++timer; rnk[timer] = u;
97         if(!son[u]) return;
98         dfs2(son[u], tp);
99         for(int v : adj[u]){
100             if(v == fa[u] || v == son[u]) continue;
101             dfs2(v, v);
102         }
103     }
104
105     //建树
106     void build(const vector<int>& a, int root = 1){
107         dfs1(root, 0, 1);
108         dfs2(root, root);
109         vector<int> v(n + 1);
110         for(int i = 1; i <= n; i++){
111             v[i] = a[rnk[i]];

```

```

111     }
112     seg.build(1, 1, n, v);
113 }
114
115 // loglog复杂度太大 可以用树上差分优化
116 void updatePath(int u, int v, int val){
117     while(top[u] != top[v]){
118         if(dep[top[u]] < dep[top[v]]) swap(u, v);
119         seg.update(1, 1, n, dfn[top[u]], dfn[u], val);
120         u = fa[top[u]];
121     }
122     if(dep[u] > dep[v]) swap(u, v);
123     seg.update(1, 1, n, dfn[u], dfn[v], val);
124 }
125
126 //询问路径
127 int askPath(int u, int v){
128     int res = 0;
129     while(top[u] != top[v]){
130         if(dep[top[u]] < dep[top[v]]) swap(u, v);
131         res += seg.ask(1, 1, n, dfn[top[u]], dfn[u]);
132         u = fa[top[u]];
133     }
134     if(dep[u] > dep[v]) swap(u, v);
135     res += seg.ask(1, 1, n, dfn[u], dfn[v]);
136     return res;
137 }
138
139 //更新树
140 void updateTree(int u, int val){
141     seg.update(1, 1, n, dfn[u], dfn[u] + sz[u] - 1, val);
142 }
143
144 //询问子树
145 int askTree(int u){
146     return seg.ask(1, 1, n, dfn[u], dfn[u] + sz[u] - 1);
147 }
148
149 //获得两点LCA
150 int getLCA(int u, int v){
151     while(top[u] != top[v]){
152         if(dep[top[u]] < dep[top[v]]) swap(u, v);
153         u = fa[top[u]];
154     }
155     return dep[u] < dep[v] ? u : v;
156 }
157
158 //获取两点间距离（无边权）
159 int getDist(int u, int v){
160     return dep[u] + dep[v] - 2 * dep[getLCA(u, v)];
161 }
162 };

```

动态维护树直径、端点、中心

```

1  #define int long long
2
3  const int MAXN=2e5+5;
4  const int LOG=32;
5

```

```

6  vector<vector<pair<int,int>>> gra(MAXN);
7  vector<vector<int>> par(MAXN,vector<int>(LOG));
8  vector<int> vis(MAXN);
9  vector<int> dep(MAXN);
10 vector<int> dis(MAXN);
11
12 void dfs(int ind){
13     for(auto [i,w]:gra[ind]){
14         if(vis[i])continue;
15         vis[i]=1;
16         par[i][0]=ind;
17         dep[i]=dep[ind]+1;
18         dis[i]=dis[ind]+w;
19         dfs(i);
20     }
21 }
22
23 int lca(int a,int b){
24     if(dep[a]>dep[b])swap(a,b);
25     for(int i=LOG-1;i>=0;i--){
26         if(dep[par[b][i]]>=dep[a]){
27             b=par[b][i];
28         }
29     }
30     if(a==b)return a;
31     for(int i=LOG-1;i>=0;i--){
32         if(par[a][i]!=par[b][i]){
33             a=par[a][i];
34             b=par[b][i];
35         }
36     }
37     return par[a][0];
38 }
39
40 int dist(int a,int b){
41     int l=lca(a,b);
42     return dis[a]+dis[b]-2*dis[l];
43 }
44
45 int jump(int x,int k){
46     for(int i=0;i<LOG;i++){
47         if(k&(1LL<<i)){
48             x=par[x][i];
49         }
50     }
51     return x;
52 }
53
54 int u_to_v_len(int u,int v,int len){
55     int l=lca(u,v);
56     int du=dist(u,l);
57     if(len<=du){
58         int cur=u;
59         for(int i=LOG-1;i>=0;i--){
60             int p=par[cur][i];
61             if(p && dist(u,p)<=len){
62                 cur=p;
63             }
64         }
65         return cur;
66     }

```

```

67     int rem=dist(u,v)-len;
68     int cur=v;
69     for(int i=LOG-1;i>=0;i--){
70         int p=par[cur][i];
71         if(p && dist(v,p)<=rem){
72             cur=p;
73         }
74     }
75     if(dist(cur,v)<rem)cur=par[cur][0];
76     return cur;
77 }
78
79 pair<int,int> get_center(int u,int v){
80     int d=dist(u,v);
81     int half=d/2;
82     int p=u_to_v_len(u,v,half);
83     if(dist(u,p)*2==d){
84         return {p,p};
85     }
86     int l=lca(u,v);
87     int nxt;
88     if(lca(u,p)==p&&p!=l){
89         int cur=u;
90         for(int i=LOG-1;i>=0;i--){
91             int pp=par[cur][i];
92             if(pp && dep[pp]>dep[p]){
93                 cur=pp;
94             }
95         }
96         nxt=par[cur][0];
97     }else{
98         int cur=v;
99         for(int i=LOG-1;i>=0;i--){
100             int pp=par[cur][i];
101             if(pp && dep[pp]>dep[p]){
102                 cur=pp;
103             }
104         }
105         nxt=cur;
106     }
107     return {p,nxt};
108 }

```

树的直径

```

1  const int N = 200005;
2  int n;
3  vector<int> g[N];
4  int dist[N];
5
6  void dfs(int u, int fa) {
7      for (int v : g[u]) {
8          if (v == fa) continue;
9          dist[v] = dist[u] + 1;
10         dfs(v, u);
11     }
12 }
13
14 int get_far(int start) {
15     memset(dist, 0, sizeof dist);

```

```

16     dfs(start, 0);
17     int far = start;
18     for (int i = 1; i <= n; i++) {
19         if (dist[i] > dist[far]) {
20             far = i;
21         }
22     }
23     return far;
24 }

```

树的重心

```

1  const int N = 200005;
2  int n;
3  vector<int> g[N];
4  int sz[N];
5  int best = INT_MAX;
6  vector<int> centroid;
7
8  void dfs(int u, int fa) {
9      sz[u] = 1;
10     int max_part = 0;
11     for (int v : g[u]) {
12         if (v == fa) continue;
13         dfs(v, u);
14         sz[u] += sz[v];
15         max_part = max(max_part, sz[v]);
16     }
17     max_part = max(max_part, n - sz[u]);
18     if (max_part < best) {
19         best = max_part;
20         centroid.clear();
21         centroid.push_back(u);
22     } else if (max_part == best) {
23         centroid.push_back(u);
24     }
25 }

```

Bellman-Ford (有边数限制最短路)

```

1  #define int long long
2  #define endl '\n'
3  const int N = 1e5 + 5;
4  const int mod = 80112002;
5  int n, m, s, k, t;
6  int x, y, z;
7  vector<pair<int, int>>g[N];
8  int dist[N];
9  int back[N];
10 bool st[N];
11 int bell()
12 {
13     memset(dist, 0x3f, sizeof dist);
14     dist[1] = 0;
15     for (int p = 0; p < k; p++)
16     {
17         memcpy(back, dist, sizeof dist);
18         for (int j = 1; j <= n; j++)
19         {

```

```

20         for (auto& t : g[j])
21         {
22             int a = t.first, b = t.second;
23             dist[a] = min(back[j] + b, dist[a]);
24         }
25     }
26 }
27 if (dist[n] > 0x3f3f3f3f3f3f3f3f / 2) return 0x3f3f3f3f3f3f3f3f;
28 else
29     return dist[n];
30 }

```

SPFA判断负环

```

1  #define int long long
2  #define endl '\n'
3  const int N = 1e4 + 5;
4  int t;
5  int n, m, u, v, w;
6  vector<pair<int,int> > arr[N];
7  int st[N], dist[N], cnt[N];
8  queue<int> q;
9  void spfa()
10 {
11     for(int i=1;i<=n;i++)
12     {
13         st[i]=1;
14         q.push(i);
15     }
16     while (!q.empty())
17     {
18         int tm = q.front();
19         q.pop();
20         st[tm] = 0;
21         for (int i = 0; i < arr[tm].size(); i++)
22         {
23             int v = arr[tm][i].first, dis = arr[tm][i].second;
24             if (dist[v] > dist[tm] + dis)
25             {
26                 dist[v] = dist[tm] + dis;
27                 if (!st[v])
28                 {
29                     cnt[v] = cnt[tm] + 1;
30                     if (cnt[v] >= n)
31                     {
32                         cout << "Yes" << endl;
33                         return;
34                     }
35                     st[v] = 1;
36                     q.push(v);
37                 }
38             }
39         }
40     }
41     cout << "No" << endl;
42 }

```

SPFA求最短路

```

1  #define int long long
2  #define endl '\n'
3  const int N = 1e5 + 5;
4  const int mod = 80112002;
5  int n, m, s;
6  int x, y, z;
7  vector<pair<int, int>>g[N];
8  int dist[N];
9  bool st[N];
10 int spfa()
11 {
12     memset(dist, 0x3f, sizeof dist);
13     dist[1] = 0;
14     queue<int> q;
15     q.push(1);
16     st[1]=true;
17     while (!q.empty())
18     {
19         auto t = q.front();
20         q.pop();
21         st[t] = false;
22         for (int i = 0; i < g[t].size(); i++)
23         {
24             if (dist[g[t][i].first] > dist[t] + g[t][i].second)
25             {
26                 dist[g[t][i].first] = dist[t] + g[t][i].second;
27                 if (!st[g[t][i].first])
28                 {
29                     q.push(g[t][i].first);
30                     st[g[t][i].first] = true;
31                 }
32             }
33         }
34     }
35     if (dist[n] == 0x3f3f3f3f3f3f3f3f) return 0x3f3f3f3f3f3f3f3f;
36     else
37         return dist[n];
38 }

```

普通Dijkstra求最短路

```

1  #define int long long
2  #define endl '\n'
3  const int N = 1e5+5;
4  const int mod = 80112002;
5  int n, m, s;
6  int x, y, z;
7  vector<pair<int, int> >g[N];
8  int dist[N];
9  bool st[N];
10 int dij()
11 {
12     memset(dist, 0x3f, sizeof dist);
13     dist[1] = 0;
14     for (int p = 0; p < n; p++)
15     {
16         int t = -1;
17         for (int j = 1; j <= n; j++)
18         {
19             if (!st[j] && (t == -1 || dist[t] > dist[j]))

```

```

20         {
21             t = j;
22         }
23     }
24     st[t] = true;
25     for (int i = 0; i < g[t].size(); i++)
26     {
27         int a = g[t][i].first, b = g[t][i].second;
28         dist[a] = min(dist[a], dist[t] + b);
29     }
30 }
31 if(dist[n]==0x3f3f3f3f3f3f3f3f) return -1;
32 else
33     return dist[n];
34 }

```

堆优化Dijkstra求最短路

```

1  #define int long long
2  #define endl '\n'
3  const int N = 1e5 + 5;
4  const int mod = 80112002;
5  int n, m, s;
6  int x, y, z;
7  vector<pair<int, int>>g[N];
8  int dist[N];
9  bool st[N];
10 int dij()
11 {
12     memset(dist, 0x3f, sizeof dist);
13     dist[1] = 0;
14     priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>
> q;
15     q.push({ 0,1 });
16     while (!q.empty())
17     {
18         auto t = q.top();
19         q.pop();
20         int dis = t.first, idx = t.second;
21         if (st[idx]) continue;
22         st[idx] = true;
23         for (int i = 0; i < g[idx].size(); i++)
24         {
25             dist[g[idx][i].second] = min(dis + g[idx][i].first, dist[g[idx]
[i].second]);
26             q.push({ dist[g[idx][i].second] ,g[idx][i].second });
27         }
28     }
29     if (dist[n] == 0x3f3f3f3f3f3f3f3f) return -1;
30     else
31         return dist[n];
32 }

```

Floyd求多源最短路

```

1  #define int long long
2  #define endl '\n'
3  int n, m, k, x, y, z;
4  const int N = 205;

```

```

5  int d[N][N];
6  void floyd()
7  {
8      for (int k = 1; k <= n; k++)
9      {
10         for (int i = 1; i <= n; i++)
11         {
12             for (int j = 1; j <= n; j++)
13             {
14                 d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
15             }
16         }
17     }
18 }

```

Prim算法求最小生成树

```

1  //define int long long
2  #define endl '\n'
3  const int N=510;
4  const int INF=0x3f3f3f3f;
5  int n,m;
6  int g[N][N];
7  int dist[N];
8  bool st[N];
9  int prim()
10 {
11     memset(dist,0x3f,sizeof dist);
12     int res=0;
13     dist[1]=0;
14     for(int i=0;i<n;i++)
15     {
16         int t=-1;
17         for(int j=1;j<=n;j++)
18         {
19             if(!st[j]&&(t==-1||dist[j]<dist[t]))
20             {
21                 t=j;
22             }
23         }
24         if(dist[t]==0x3f3f3f3f)
25         {
26             return INF;
27         }
28         res+=dist[t];
29         st[t]=true;
30         for(int j=1;j<=n;j++)
31         {
32             if(!st[j])
33                 dist[j]=min(dist[j],g[t][j]);
34         }
35     }
36     return res;
37 }

```

Kruskal算法求最小生成树

```

1  #define int long long
2  #define endl '\n'

```

```

3  const int N=2e5+10;
4  int t,n,m,res,cnt;
5  int fa[N];
6  struct edge
7  {
8      int a,b,w;
9      bool operator<(edge& x)
10     {
11         return w<x.w;
12     }
13 }e[N];
14
15 int find(int u)
16 {
17     if(u!=fa[u])
18     {
19         fa[u]=find(fa[u]);
20     }
21     return fa[u];
22 }

```

拓扑排序

```

1  #define int long long
2  #define endl '\n'
3  const int N = 1e5 + 5;
4  int n, m;
5  vector<int> arr;
6  vector<int> g[N];
7  int d[N];
8  queue<int> q;
9
10 bool top()
11 {
12     for (int i = 1; i <= n; i++)
13     {
14         if (d[i] == 0)
15         {
16             q.push(i);
17             arr.push_back(i);
18         }
19     }
20     while (!q.empty())
21     {
22         int t = q.front();
23         q.pop();
24         for (int i = 0; i < g[t].size(); i++)
25         {
26             d[g[t][i]]--;
27             if (d[g[t][i]] == 0)
28             {
29                 q.push(g[t][i]);
30                 arr.push_back(g[t][i]);
31             }
32         }
33     }
34     if (arr.size() == n) return true;
35     else return false;
36 }

```

匈牙利二分图最大匹配

```
1  #define int long long
2  #define endl '\n'
3  const int MAXN=505;
4  vector<int> state;
5  vector<int> vis(MAXN);
6  vector<vector<int>> vec;
7  int visnum=1;
8  int check(int ind){
9      for(int i:vec[ind]){
10         if(vis[i]<visnum){
11             vis[i]=visnum;
12             if(state[i]==0||check(state[i])){
13                 state[i]=ind;
14                 return 1;
15             }
16         }
17     }
18     return 0;
19 }
20
21 /*
22 补充定义和定理：
23
24 最大匹配数：最大匹配的匹配边的数目
25
26 最小点覆盖数：选取最少的点，使任意一条边至少有一个端点被选择
27
28 最大独立集：选取最多的点，使任意所选两点均不相连
29
30 最小路径覆盖数：对于一个 DAG（有向无环图），选取最少条路径，使得每个顶点属于且仅属于一条路径。路径长可以为 0（即单个点）。
31
32 定理1：最大匹配数 = 最小点覆盖数（这是 konig 定理）
33
34 定理2：最大独立集 = 顶点数 - 最大匹配数
35
36 定理3：最小路径覆盖数 = 顶点数 - 最大匹配数
37
38 */
```

带权二分图最优匹配（KM算法）

```
1  #define int long long
2
3  const int inf=0x7fffffff;
4  const int MAXN=505;
5
6  int cnta,cntb;
7  int w[MAXN][MAXN];
8
9  int visa[MAXN],visb[MAXN];
10 int la[MAXN],lb[MAXN];
11 int match[MAXN],upd[MAXN];
12 int visnum=0;
13 bool check(int x) {
14     visa[x]=visnum;
15     for(int y=1; y<=cnta; y++){
```

```

16         if(w[x][y]!=-inf){
17             if(visb[y]<visnum)
18                 if(la[x]+lb[y]==w[x][y]) {
19                     visb[y]=visnum;
20                     if(!match[y]||check(match[y])) {
21                         match[y]=x;
22                         return true;
23                     }
24                 } else upd[y]=min(upd[y],la[x]+lb[y]-w[x][y]);
25             }
26         }
27     return false;
28 }
29
30 int KM() {
31     memset(match,0,sizeof(match));
32     for(int i=1; i<=cnta; i++) {
33         la[i]=-inf;
34         lb[i]=0;
35         for(int j=1; j<=cntb; j++){
36             la[i]=max(la[i],w[i][j]);
37         }
38     }
39     for(int i=1; i<=cnta; i++){
40         while(true) {
41             visnum++;
42             for(int j=1; j<=cntb; j++){
43                 upd[j]=inf;
44             }
45             if(check(i))break;
46             int delta=inf;
47             for(int j=1; j<=cntb; j++){
48                 if(visb[j]<visnum)delta=min(delta,upd[j]);
49             }
50             for(int j=1; j<=cntb; j++) {
51                 if(visa[j]==visnum)la[j]-=delta;
52                 if(visb[j]==visnum)lb[j]+=delta;
53             }
54         }
55     }
56     int ans=0;
57     for(int i=1; i<=cntb; i++){
58         ans+=w[match[i]][i];
59     }
60     return ans;
61 }
62
63 /*
64     如果要求完美匹配的最小权，将边权取负数w[i][j]=-w[i][j]
65     最后答案输出-KM()即可
66 */

```

染色法判定二分图

```

1  #define int long long
2  #define endl '\n'
3  const int N=1e5+10;
4  int t,n,m;
5  vector<int> arr[N];
6  int color[N]; // 1 / 2

```

```

7  bool dfs(int u,int c)
8  {
9      color[u]=c;
10     for(auto x:arr[u])
11     {
12         if(!color[x])
13         {
14             if(!dfs(x,3-c)) return 0;
15         }
16         else if(color[x]&&color[x]!=3-c)
17         {
18             return 0;
19         }
20     }
21     return 1;
22 }

```

网络流

Dinic最大流

```

1  using ll = long long;
2  const ll INF = 4e18;
3
4  struct Dinic {
5      struct Edge {
6          int to, rev;
7          ll cap;
8      };
9
10     int n;
11     vector<vector<Edge>> g;
12     vector<int> level, cur;
13
14     Dinic(int n) : n(n), g(n), level(n), cur(n) {}
15
16     void addEdge(int u, int v, ll cap) {
17         Edge a{v, (int)g[v].size(), cap};
18         Edge b{u, (int)g[u].size(), 0};
19         g[u].push_back(a);
20         g[v].push_back(b);
21     }
22
23     bool bfs(int s, int t) {
24         fill(level.begin(), level.end(), -1);
25         queue<int> q;
26         level[s] = 0;
27         q.push(s);
28         while (!q.empty()) {
29             int u = q.front();
30             q.pop();
31             for (auto &e : g[u]) {
32                 if (e.cap > 0 && level[e.to] == -1) {
33                     level[e.to] = level[u] + 1;
34                     q.push(e.to);
35                 }
36             }
37         }
38         return level[t] != -1;

```

```

39     }
40
41     ll dfs(int u, int t, ll flow) {
42         if (u == t) return flow;
43         for (int &i = cur[u]; i < (int)g[u].size(); i++) {
44             Edge &e = g[u][i];
45             if (e.cap > 0 && level[e.to] == level[u] + 1) {
46                 ll f = dfs(e.to, t, min(flow, e.cap));
47                 if (f > 0) {
48                     e.cap -= f;
49                     g[e.to][e.rev].cap += f;
50                     return f;
51                 }
52             }
53         }
54         return 0;
55     }
56
57     ll maxflow(int s, int t) {
58         ll flow = 0;
59         while (bfs(s, t)) {
60             fill(cur.begin(), cur.end(), 0);
61             while (ll f = dfs(s, t, INF)) {
62                 flow += f;
63             }
64         }
65         return flow;
66     }
67 };

```

EK最大流

```

1  using ll = long long;
2  const ll INF = 4e18;
3
4  struct EdmondsKarp {
5      struct Edge {
6          int to, rev;
7          ll cap;
8      };
9
10     int n;
11     vector<vector<Edge>> g;
12
13     EdmondsKarp(int n) : n(n), g(n) {}
14
15     void addEdge(int u, int v, ll cap) {
16         Edge a{v, (int)g[v].size(), cap};
17         Edge b{u, (int)g[u].size(), 0};
18         g[u].push_back(a);
19         g[v].push_back(b);
20     }
21
22     ll maxflow(int s, int t) {
23         ll flow = 0;
24         while (true) {
25             vector<int> prev(n, -1);
26             vector<int> preE(n, -1);
27             vector<ll> mf(n, 0);
28             queue<int> q;

```

```

29         q.push(s);
30         mf[s] = INF;
31         while (!q.empty() && mf[t] == 0) {
32             int u = q.front();
33             q.pop();
34             for (int i = 0; i < (int)g[u].size(); i++) {
35                 Edge &e = g[u][i];
36                 if (e.cap > 0 && mf[e.to] == 0) {
37                     mf[e.to] = min(mf[u], e.cap);
38                     prev[e.to] = u;
39                     preE[e.to] = i;
40                     q.push(e.to);
41                     if (e.to == t)
42                         break;
43                 }
44             }
45         }
46         if (mf[t] == 0)
47             break;
48         ll f = mf[t];
49         flow += f;
50         int v = t;
51         while (v != s) {
52             int u = prev[v];
53             int id = preE[v];
54             Edge &e = g[u][id];
55             e.cap -= f;
56             g[v][e.rev].cap += f;
57             v = u;
58         }
59     }
60     return flow;
61 }
62 };

```

Dinic最小割

```

1  using ll = long long;
2  const ll INF = 4e18;
3
4  struct Dinic {
5      struct Edge {
6          int to, rev;
7          ll cap;
8          ll originalCap;
9      };
10
11     int n;
12     vector<vector<Edge>> g;
13     vector<int> level, cur;
14
15     Dinic(int n) : n(n), g(n), level(n), cur(n) {}
16
17     void addEdge(int u, int v, ll cap) {
18         Edge a{v, (int)g[v].size(), cap, cap};
19         Edge b{u, (int)g[u].size(), 0, 0};
20         g[u].push_back(a);
21         g[v].push_back(b);
22     }
23

```

```

24     bool bfs(int s, int t) {
25         fill(level.begin(), level.end(), -1);
26         queue<int> q;
27         q.push(s);
28         level[s] = 0;
29         while (!q.empty()) {
30             int u = q.front();
31             q.pop();
32             for (auto &e : g[u]) {
33                 if (e.cap > 0 && level[e.to] == -1) {
34                     level[e.to] = level[u] + 1;
35                     q.push(e.to);
36                 }
37             }
38         }
39         return level[t] != -1;
40     }
41
42     ll dfs(int u, int t, ll flow) {
43         if (u == t)
44             return flow;
45         for (int &i = cur[u]; i < (int)g[u].size(); i++) {
46             Edge &e = g[u][i];
47             if (e.cap > 0 && level[e.to] == level[u] + 1) {
48                 ll f = dfs(e.to, t, min(flow, e.cap));
49                 if (f > 0) {
50                     e.cap -= f;
51                     g[e.to][e.rev].cap += f;
52                     return f;
53                 }
54             }
55         }
56         return 0;
57     }
58
59     ll maxflow(int s, int t) {
60         ll flow = 0;
61         while (bfs(s, t)) {
62             fill(cur.begin(), cur.end(), 0);
63             while (ll f = dfs(s, t, INF)) {
64                 flow += f;
65             }
66         }
67         return flow;
68     }
69
70     // 最大流跑完后调用
71     // vis[i] = 1: 最小割 S 侧
72     // vis[i] = 0: 最小割 T 侧
73     vector<int> mincut(int s) {
74         vector<int> vis(n, 0);
75         queue<int> q;
76         vis[s] = 1;
77         q.push(s);
78         while (!q.empty()) {
79             int u = q.front();
80             q.pop();
81             for (auto &e : g[u]) {
82                 if (e.cap > 0 && !vis[e.to]) {
83                     vis[e.to] = 1;
84                     q.push(e.to);

```

```

85         }
86     }
87 }
88 return vis;
89 }
90 };

```

最小费用最大流

```

1  using ll = long long;
2  const ll INF = 4e18;
3
4  struct MinCostMaxFlow {
5      struct Edge {
6          int to, rev;
7          ll cap, cost;
8      };
9
10     int n;
11     vector<vector<Edge>> g;
12
13     MinCostMaxFlow(int n) : n(n), g(n) {}
14
15     void addEdge(int u, int v, ll cap, ll cost) {
16         Edge a{v, (int)g[v].size(), cap, cost};
17         Edge b{u, (int)g[u].size(), 0, -cost};
18         g[u].push_back(a);
19         g[v].push_back(b);
20     }
21
22     // 返回 {最大流, 最小费用}
23     pair<ll, ll> minCostMaxFlow(int s, int t) {
24         ll maxFlow = 0;
25         ll minCost = 0;
26         vector<ll> dist(n);
27         vector<int> inq(n);
28         vector<int> prev(n), preE(n);
29         while (true) {
30             fill(dist.begin(), dist.end(), INF);
31             fill(inq.begin(), inq.end(), 0);
32             queue<int> q;
33             dist[s] = 0;
34             q.push(s);
35             inq[s] = 1;
36             while (!q.empty()) {
37                 int u = q.front();
38                 q.pop();
39                 inq[u] = 0;
40                 for (int i = 0; i < (int)g[u].size(); i++) {
41                     Edge &e = g[u][i];
42                     if (e.cap > 0 &&
43                         dist[e.to] > dist[u] + e.cost) {
44                         dist[e.to] = dist[u] + e.cost;
45                         prev[e.to] = u;
46                         preE[e.to] = i;
47                         if (!inq[e.to]) {
48                             inq[e.to] = 1;
49                             q.push(e.to);
50                         }
51                     }
52                 }
53             }
54         }
55     }
56 }

```

```

52         }
53     }
54     if (dist[t] == INF)
55         break;
56     ll f = INF;
57     for (int v = t; v != s; v = prev[v]) {
58         int u = prev[v];
59         int id = preE[v];
60         f = min(f, g[u][id].cap);
61     }
62     for (int v = t; v != s; v = prev[v]) {
63         int u = prev[v];
64         int id = preE[v];
65         Edge &e = g[u][id];
66         e.cap -= f;
67         g[v][e.rev].cap += f;
68     }
69     maxFlow += f;
70     minCost += f * dist[t];
71 }
72 return {maxFlow, minCost};
73 }
74 };

```

带费用上下界最小费用可行循环流

```

1  using ll = long long;
2  const ll INF = (1LL << 60);
3
4  /*=====
5      普通最小费用最大流
6  =====*/
7
8  struct MinCostMaxFlow {
9      struct Edge {
10         int to, rev;
11         ll cap, cost;
12     };
13
14     int n;
15     vector<vector<Edge>> g;
16
17     MinCostMaxFlow(int n) : n(n), g(n) {}
18
19     int addEdge(int u, int v, ll cap, ll cost) {
20         int id = g[u].size();
21         Edge a{v, (int)g[v].size(), cap, cost};
22         Edge b{u, (int)g[u].size(), 0, -cost};
23         g[u].push_back(a);
24         g[v].push_back(b);
25         return id;
26     }
27
28     pair<ll, ll> minCostMaxFlow(
29         int s,
30         int t,
31         ll limit = INF
32     ) {
33         ll flow = 0;
34         ll cost = 0;

```

```

35     vector<ll> dist(n);
36     vector<int> inq(n);
37     vector<int> prev(n), preE(n);
38     while (flow < limit) {
39         fill(dist.begin(), dist.end(), INF);
40         fill(inq.begin(), inq.end(), 0);
41         queue<int> q;
42         dist[s] = 0;
43         q.push(s);
44         inq[s] = 1;
45         while (!q.empty()) {
46             int u = q.front();
47             q.pop();
48             inq[u] = 0;
49             for (int i = 0; i < (int)g[u].size(); i++) {
50                 Edge &e = g[u][i];
51                 if (e.cap > 0 &&
52                     dist[e.to] > dist[u] + e.cost) {
53                     dist[e.to] = dist[u] + e.cost;
54                     prev[e.to] = u;
55                     preE[e.to] = i;
56                     if (!inq[e.to]) {
57                         inq[e.to] = 1;
58                         q.push(e.to);
59                     }
60                 }
61             }
62         }
63         if (dist[t] == INF)
64             break;
65         ll f = limit - flow;
66         for (int v = t; v != s; v = prev[v]) {
67             int u = prev[v];
68             int id = preE[v];
69             f = min(f, g[u][id].cap);
70         }
71         for (int v = t; v != s; v = prev[v]) {
72             int u = prev[v];
73             int id = preE[v];
74             Edge &e = g[u][id];
75             e.cap -= f;
76             g[v][e.rev].cap += f;
77         }
78         flow += f;
79         cost += f * dist[t];
80     }
81     return {flow, cost};
82 }
83 };
84
85
86 /*=====
87     带费用上下界最小费用可行循环流
88
89     addEdge(u, v, lower, upper, cost)
90
91     solve():
92         返回 {是否存在可行流, 最小费用}
93     =====*/
94
95 struct LowerBoundMinCostFlow {

```

```

96
97     struct OriginalEdge {
98         int u;
99         int id;
100         ll lower;
101         ll upper;
102     };
103
104     int n;
105     int SS, TT;
106     MinCostMaxFlow mf;
107     vector<ll> balance;
108     ll baseCost = 0;
109     vector<OriginalEdge> edges;
110
111     LowerBoundMinCostFlow(int n)
112         : n(n),
113           SS(n),
114           TT(n + 1),
115           mf(n + 2),
116           balance(n, 0) {}
117
118     int addEdge(
119         int u,
120         int v,
121         ll lower,
122         ll upper,
123         ll cost
124     ) {
125         baseCost += lower * cost;
126         balance[u] -= lower;
127         balance[v] += lower;
128         int id = mf.addEdge(
129             u,
130             v,
131             upper - lower,
132             cost
133         );
134         edges.push_back({
135             u,
136             id,
137             lower,
138             upper
139         });
140         return (int)edges.size() - 1;
141     }
142
143     pair<bool, ll> solve() {
144         ll need = 0;
145         for (int i = 0; i < n; i++) {
146             if (balance[i] > 0) {
147                 mf.addEdge(
148                     SS,
149                     i,
150                     balance[i],
151                     0
152                 );
153                 need += balance[i];
154             }
155             else if (balance[i] < 0) {
156                 mf.addEdge(

```

```

157         i,
158         TT,
159         -balance[i],
160         0
161     );
162 }
163 }
164 auto [flow, cost]
165     = mf.minCostMaxFlow(SS, TT, need);
166 if (flow != need) {
167     return {false, 0};
168 }
169 return {
170     true,
171     baseCost + cost
172 };
173 }
174
175 ll getFlow(int edgeId) {
176     auto &x = edges[edgeId];
177     ll remain = mf.g[x.u][x.id].cap;
178     ll extra =
179         (x.upper - x.lower) - remain;
180     return x.lower + extra;
181 }
182 };

```